2017 IEEE International Conference on Information Reuse and Integration

On the Synthesis of Guaranteed-Quality Plans for Robot Fleets in Logistics Scenarios via Optimization Modulo Theories

Francesco Leofante
Erika Ábrahám
RWTH Aachen University

Tim Niemueller
Gerhard Lakemeyer
RWTH Aachen University

Armando Tacchella
University of Genoa

Abstract

In manufacturing, the increasing involvement of autonomous robots in production processes poses new challenges on the production management. In this paper we report on the usage of Optimization Modulo Theories (OMT) to solve certain multi-robot scheduling problems in this area. Whereas currently existing methods are heuristic, our approach guarantees optimality for the computed solution. We do not only present our final method but also its chronological development, and draw some general observations for the development of OMT-based approaches.

1 Introduction

With the advent of Industry 4.0, factories are moving from static process chains towards more autonomy, by introducing robots in their production lines. Given this paradigm shift, the problem of managing and optimizing the in-factory supply chain carried out by (fleets of) autonomous robots becomes crucial.

The RoboCup Logistics League (RCLL) [13] has been proposed as a realistic testbed to study the above mentioned problem at a comprehensible and manageable scale. There, groups of robots need to maintain and optimize the material flow according to dynamic orders in a simplified factory environment.

Though there exist successful symbolic reasoning methods towards solving the underlying scheduling problem [9, 12], a disadvantage of these methods is that they provide no guarantees about the quality of the solution. A promising solution to this problem is offered by the recently emerging field of Optimization Modulo Theories (OMT), where Satisfiability Modulo Theories (SMT) solving is extended with functionalities towards optimization [15, 18, 20].

In this paper we report on the application of OMT to compute optimal strategies for multi-robot systems within the RCLL scope. We encode the underlying scheduling problem as a linear mixed-integer problem which can be solved by OMT solvers such as `Z3` [7], `SMT-RAT` [5] and `OptiMathSAT` [19].

We integrated our OMT-based planning module in the RCLL planning framework [14]. By rigorous experimental analysis we show that naive encodings fail to cope with the complexity of the domain. We then detail our findings and solutions adopted to overcome previous limitations.

After presenting some preliminaries in Section 2, we specify the problem we are going to solve in Section 3. We explain our OMT-based solution and provide experimental evaluations in Section 4. Finally, we draw some general conclusions and discuss future directions of research in the Sections 5 and 6.

2 Preliminaries

Mixed-integer arithmetic. Problems considered in this work are encoded as *mixed-integer arithmetic* formulas. Syntactically, (arithmetic) *terms* are constant symbols, variables, and sums, differences or products of terms. (Arithmetic) *constraints* compare two arithmetic terms using $<$, $\leq$, $=$, $\geq$ or $>$. (Quantifier-free arithmetic) *formulas* use conjunction $\wedge$ and negation $\neg$ (and further syntactic sugar like disjunction $\vee$ or implication $\rightarrow$) to combine theory constraints. Formulas that do not use multiplication are called *linear*. A formula in *conjunctive normal form* (*CNF*) is a conjunction of disjunctions of theory constraints or negated theory constraints (see Eq. (1) for a simple example formula in CNF). Semantically, each variable is interpreted over either the real or the integer domain by an *assignment*, assigning to each variable a value from its domain; we use the standard semantics to evaluate formulas.

Optimization modulo theories. *Satisfiability modulo theories* (*SMT*) *solving* aims at deciding the satisfiability of (usually quantifier-free) first-order logic formulas over some theories like, *e.g.*, the theories of lists, arrays, bit vectors, real or (mixed-)integer arithmetic. To decide the sat-

978-1-5386-1562-1/17 $31.00 © 2017 IEEE
DOI 10.1109/IRI.2017.67

IEEE computer society

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.

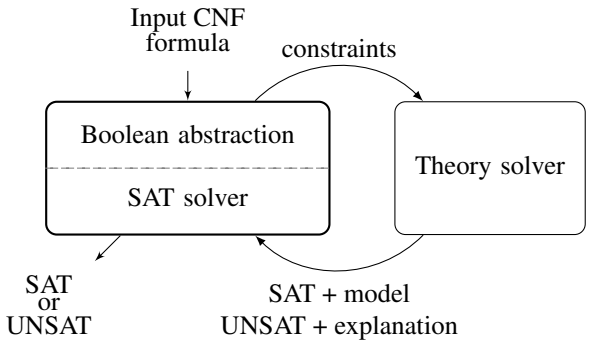


Figure 1: The SMT solving framework.

isfiability of an input formula φ in CNF, SMT solvers (see Fig. 1) typically first build a *Boolean abstraction* $abs(\varphi)$ of φ by replacing each constraint by a fresh Boolean variable (proposition), *e.g.*,

$$\begin{array}{rl} \varphi & = x \geq y \wedge (\ y > 0 \vee x > 0\ ) \wedge y \leq 0 \\ abs(\varphi) = & A \quad \wedge (\quad B \quad \vee \quad C \quad) \wedge \ \neg B \end{array} \tag{1}$$

where x and y are real-valued variables, and A, B and C are propositions.

A *Boolean satisfiability* (*SAT*) *solver* searches for a satisfying assignment S for $abs(\varphi)$, *e.g.*, $S(A) = 1$, $S(B) = 0$, $S(C) = 1$ for the above example. If no such assignment exists then the input formula φ is unsatisfiable. Otherwise, the consistency of the assignment in the underlying theory is checked by a *theory solver*. In our example, we check whether the set $\{x \geq y,\ y \leq 0,\ x > 0\}$ of linear inequalities is feasible, which is the case. If the constraints are consistent then a satisfying solution (*model*) is found for φ. Otherwise, the theory solver returns a theory lemma φ_E giving an *explanation* for the conflict, *e.g.*, the negated conjunction some inconsistent input constraints. The explanation is used to refine the Boolean abstraction $abs(\varphi)$ to $abs(\varphi) \wedge abs(\varphi_E)$. These steps are iteratively executed until either a theory-consistent Boolean assignment is found, or no more Boolean satisfying assignments exist. *Less lazy* variants invoke theory checks more frequently also on partial assignments.

Optimization modulo theories (*OMT*) [3, 5, 19, 20] extends SMT solving with optimization procedures to find a variable assignment that defines an optimal (say minimal) value for an objective function f (or a linear, lexicographic, Pareto, etc. combination of multiple objective functions) under all models of a formula φ. Most OMT solvers implement the *linear-search* scheme [18], which first uses SMT solving to determine a model S for φ. Let φ_S be the conjunction of all theory constraints that are true under S and the negation of those that are false under S. In minimization problems, a minimum μ for f is computed under the side condition φ_S, and φ is updated to

$$\varphi \wedge (f < \mu) \wedge \neg\varphi_S \tag{2}$$

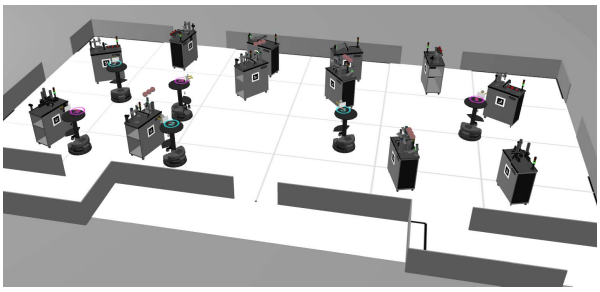


Figure 2: The simulated RCLL environment.

Repeating this procedure until the formula becomes unsatisfiable will lead to an assignment minimizing f under all models of φ.

Planning with SMT solving. SMT solvers are nowadays embedded as core engines in a wide range of technologies (see *e.g.* [1] for some examples). In the area of planning, [10] and [21] use SMT solving to generate task and motion plans. The authors of [6] perform task and motion planning which leverages incremental solving in `Z3` to update constraints about motion feasibility. The work [16] presents a motion planning framework where SMT solving is used to combine motion primitives so that they satisfy some linear temporal logic (LTL) requirements.

In contrast to the above works, (i) we do not use additional knowledge (*e.g.*, motion primitives, plan outlines) to seed the search performed by the SMT solver and (ii) we exploit OMT solving to synthesize plans that are not only *feasible* but also *optimal*.

3 Logistics Robots in Simulation

We consider a part of the problem posed to the participants of the *Planning Competition for Logistics Robots in Simulation* [11] of the RoboCup Logistics League. The simulated environment is illustrated in Fig. 2. Each game involves two teams, consisting of three robots each. The teams do not need to interact with each other but they execute on the same field. There are 12 machines placed on the field. Each team owns 6 of the machines on which they need to operate to produce products according to some dynamically issued orders.

The field is divided into 24 zones, 12 per team. Each team knows that 6 of its 12 zones contain a machine and the other 6 have no machines in it. Before the production can start, each team needs to *explore* its assigned zones in order to identify the locations and the orientations of their machines. The exploration must be completed within 20 minutes.

In this work we focus on the above-described exploration

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.

phase. In this phase, we need to compute a *plan*, which assigns to each of the three robots of a team a sequence of zones to be visited, such that each of the 12 team zones is visited by exactly one of the robots. For a successful exploration, the computation and the execution of these plans must be completed within the deadline. Therefore, the goal is to identify plans with sufficiently short (in best case *minimal*) total execution time.

The problem formulation is simple, but its solution is highly challenging (note that the problem is a variation of the multiple traveling salesman problem). Currently used solutions are heuristic and cannot assure that the computed solutions are optimal. Even if such plans allow to complete the exploration before the deadline, optimal plans might terminate considerably before the deadline and thereby allow to reduce energy consumption and give the team more time to prepare for the following production phase of the game.

Our aim is to analyze the applicability of OMT solving in this context, *i.e.*, to analyze whether optimal solutions can be determined using OMT solving in a sufficiently short time, such that the computed plan can still be safely executed in the remaining time before the exploration deadline.

4 OMT-based Planning

The experimental analysis that follows has been carried out using `Z3`, a well-established OMT solver, on a machine running Ubuntu Mate 16.4, Intel Core i7 CPU at 2.10GHz and 8GB of RAM. We used the `Python` API of `Z3` in most cases, with a single exception of the model validation in Section 5, where we used the `C++` API to integrate our OMT-based solution into the RCLL framework.

A: First encoding. We encode the planning task from Section 3 to explore Z zones by 3 robots. Robots start from a depot, modeled by some *fictitious zones* $-3, -2, -1$. Each robot $i \in \{1, 2, 3\}$ starts at zone $-i$, moves over to the zones $-i{+}1, \ldots, 0$, and explores, from the *start zone* 0, at most Z of the zones $1, \ldots, Z$. The distance between two zones i and j is denoted by $D(i, j)$. Here we assume the distance that a robot needs to travel to reach the start zone to be 0, but it could be also set to any positive value (see Fig. 3).

The movements of robot i are encoded by a sequence $pos_{i,-i}, \ldots, pos_{i,Z}$ of zones it should visit, with $pos_{i,j} \in \mathbb{Z}$. The variables $pos_{i,-i}, \ldots, pos_{i,0}$ represent the movements from the depot to the start zone:

$$\begin{aligned} pos_{1,-1} = -1 \wedge pos_{1,0} = 0 \wedge \\ pos_{2,-2} = -2 \wedge pos_{2,-1} = -1 \wedge pos_{2,0} = 0 \wedge \\ pos_{3,-3} = -3 \wedge pos_{3,-2} = -2 \wedge pos_{3,-1} = -1 \wedge pos_{3,0} = 0 \end{aligned} \tag{3}$$

For $j > 0$, if the value of $pos_{i,j}$ is between 1 and Z then it encodes the jth zone visited by robot i. Otherwise, $pos_{i,j} = -4$ encodes that the robot stopped moving and

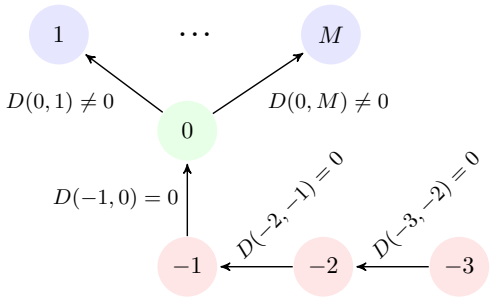


Figure 3: Initial robot configuration.

stays at $pos_{i,j-1}$ for the rest of the exploration (*i.e.*, the plan does not require robot i to explore any more zones). The total distance traveled by robot i to visit zones until step j is stored in $d_{i,j} \in \mathbb{R}$. These facts are encoded for each robot $i \in \{1, 2, 3\}$ by $d_{i,0} = 0$ and for each $j \in \{1, \ldots, Z\}$ by:

$$\begin{aligned} \bigvee_{k=0}^{Z} \bigvee_{\substack{l=1 \\ l \neq k}}^{Z} & \Big(pos_{i,j-1} = k \wedge pos_{i,j} = l \wedge d_{i,j} = d_{i,j-1} + D(k,l) \Big) \vee \\ & \Big(pos_{i,j} = -4 \wedge d_{i,Z} = d_{i,j-1} \Big) \end{aligned} \tag{4}$$

which ensures that, at each step j, either the robot moves and its travel distance is incremented accordingly, or the robot stops moving. Notice that in this second case, we can immediately determine the final travel distance for the robot at the last step in the plan and, furthermore, the above constraints imply that once a robot stops moving it will not move in the future ($pos_{i,j} = -4$ implies $pos_{i,j'} = -4$ for all $j' > j$).

For each zone $k \in \{1, \ldots, Z\}$ we enforce that it is visited exactly once by requiring:

$$\bigvee_{i=1}^{3} \bigvee_{j=1}^{Z} \Big(pos_{i,j} = k \wedge \bigwedge_{u=1}^{3} \bigwedge_{\substack{v=1 \\ (v,u) \neq (i,j)}}^{Z} pos_{u,v} \neq k \Big) \tag{5}$$

Finally, for each robot $i \in \{1, 2, 3\}$ we introduce the Boolean variable m_i to encode whether the total travel distance $d_{i,Z}$ for the robot at the end of its plan is the maximum over all total travel distances:

$$m_i \Leftrightarrow \Big(\bigwedge_{\substack{l=1 \\ l<i}}^{3} d_{l,Z} < d_{i,Z} \wedge \bigwedge_{\substack{l=1 \\ i<l}}^{3} d_{l,Z} \leq d_{i,Z} \Big) \tag{6}$$

Our optimization objective is to minimize the largest total travel distance:

$$\text{minimize} \quad \sum_{i=1}^{3} m_i \cdot d_{i,Z} \tag{7}$$

Results. We consider four benchmarks with 6, 8, 10 and 12 zones to be visited. Encoding A allowed us to compute optimal plans, but it does not scale with the number of zones to be visited. The solving time 286.7 seconds listed in Table 1 for the optimal objective 12.6 for a benchmark

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.

Z	A		B		C		D		E		F		Optimum
	Time	Conf	Time	Conf	Time	Conf	Time	Conf	Time	Conf	Time	Conf	
6	0.40	4841	0.25	3206	0.18	2525	0.17	2069	0.29	3416	0.16	1103	10.9
8	2.07	14400	1.91	15248	1.16	9237	1.62	14355	5.32	30302	1.23	3876	11.4
10	80.06	225518	59.71	184685	26.71	91648	21.72	89785	TO		8.97	27811	12.1
12	286.70	486988	255.55	449485	81.64	198249	54.17	161134	TO		36.21	101308	12.6

Table 1: Running times (sec) and #conflicts for encodings A-F (Z: number of zones to be visited, TO: 5min).

with $Z = 12$ zones claims a large part of the overall duration of the exploration phase.

B: Tackling loosely connected constraints. By analyzing solver statistics we noticed that the number of theory conflicts was quite large, and theory conflicts typically appeared at relatively high decision levels, *i.e.*, at late stages of the Boolean search in the SAT solver. One reason for this is that during optimization, violations of upper bounds on the total travel distances can be recognized by the theory solver only if all the zones that a robot should visit are already decided. In other words, the constraints defining the total travel distance of a robot build a loosely connected chain in their variable-dependency graph. Furthermore, explanations of the theory conflicts blamed the whole plan of a robot, instead of restricting it to prefixes that already lead to violation. As a result, the propositional search tree could not be efficiently pruned. To alleviate this problem, we added to the encoding A for all $i \in \{1, 2, 3\}$ the following formula, which is implied by the monotone increment of the partial travel distances by further zone visits:

$$\bigwedge_{j=1}^{Z} d_{i,j} \leq d_{i,Z} \tag{8}$$

Results. As Table 1 shows, adding the above constraints led to a slight improvement, but the solving time of 255.55 seconds for 12 zones is still too long for our application.

C: Symmetry breaking. Although the robots start from different zones, all move to the start location 0 at cost 0 before exploration. Thus, given a schedule for the three robots, a renaming of the robots gives another schedule with the same maximal travel distance. These symmetries result in the solver covering unnecessarily redundant search space, significantly increasing solving time. However, breaking these symmetries by modifying the encoding and without modifying the solver-internal algorithms is hard. A tiny part of these symmetries, however, can be broken by imposing on top of encoding B that a single, heuristically determined zone k (*e.g.*, the closest or furthest to zone 0) should be visited by a robot i:

$$\bigvee_{j=1}^{Z} pos_{i,j} = k \tag{9}$$

Results. This at first sight rather weak symmetry-breaking formula proved to be beneficial, resulting in a greatly reduced number of conflicts as well as solving time (81.64 seconds for $Z = 12$ zones, see Table 1). However, this encoding just fixes the robot that should visit a given single zone, thus the computational effort for Z zones reduces only to a value comparable to the previous effort (using encoding B) for $Z - 1$.

D: Explicit scheduler choice. In order to make the domain over which the variables $pos_{i,j}$ range more explicit, we added to encoding C the following constraints for all $i \in \{1, 2, 3\}$ and $j \in \{1, \ldots, Z\}$:

$$pos_{i,j} = -4 \vee \bigvee_{k=1}^{Z} pos_{i,j} = k \tag{10}$$

Results. This addition led to some performance gain. With a solving time of 54.17 seconds for 12 zones, our approach could be successfully integrated in the RCCL planning framework.

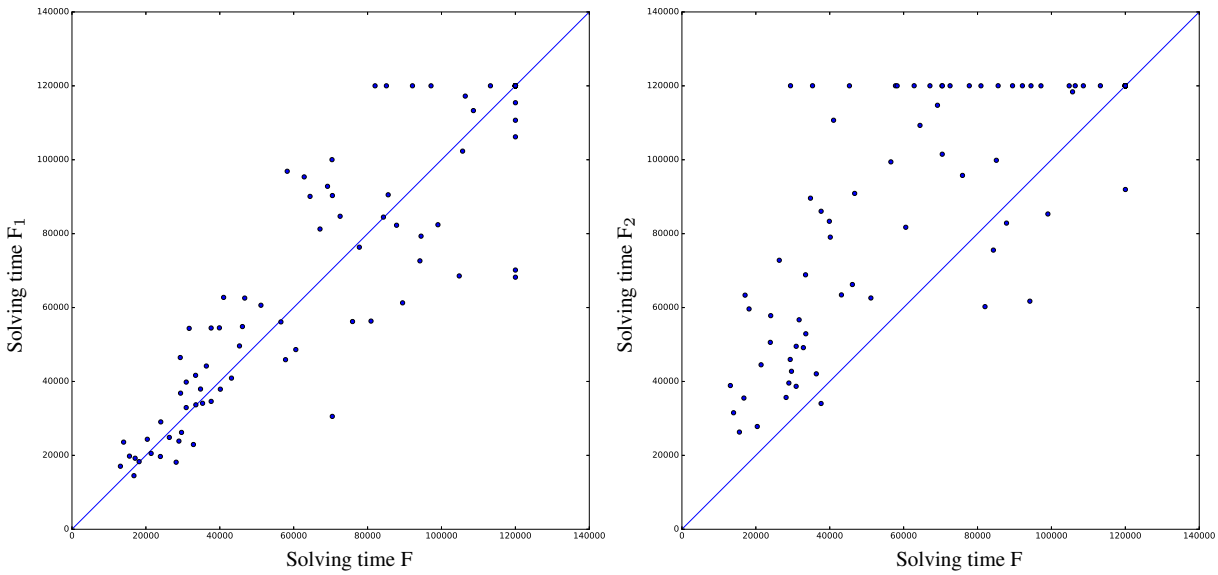
E: Partial bit-blasting. To reduce the number and size of theory checks, we also experimented with partial bit-blasting: the theory constraints $pos_{i,j}{=}k$ in encoding C were replaced by Boolean propositions $pos_{i,j,k}{\in}\mathbb{B}$, which are true iff robot i visits zone k at step j. For each $i{\in}\{1, 2, 3\}$ and $j{\in}\{-3, \ldots, Z\}$ we ensure that there is exactly one $k{\in}\{-4, \ldots, Z\}$ for which $pos_{i,j,k}$ is true by bit-blasting for the $Z{+}5$ possible values (using fresh propositions $p_{i,j,k} \in \mathbb{B}$):

$$\begin{aligned} pos_{i,j,0} &\iff (\neg p_{i,j,\lceil \log(Z+5) \rceil} \wedge \ldots \wedge \neg p_{i,j,0}) \\ pos_{i,j,1} &\iff (\neg p_{i,j,\lceil \log(Z+5) \rceil} \wedge \ldots \wedge \ p_{i,j,1}) \end{aligned} \quad \ldots (11)$$

Results. As shown in Table 1, partial bit-blasting did not introduce any improvement. On the contrary, an optimal solution for 12 zones could not be computed within 5 minutes. We made several other attempts to improve the running times by modifying encoding D, but they did not bring any major improvement.

F: Explicit decisions. Even though encoding D could be integrated in the RCLL framework, we investigated ways to further reduce the solving times.

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Figure 4: Comparison of solving times (msec) for encodings F, F_1 and F_2 (TO: 2min).

To this purpose, we developed a new encoding, in which we made some decisions explicit by means of additional variables. In particular, we introduced integer variables m_k to encode which robot visits zone k, and integer variables $n_{i,k}$ to count how many of the zones $1, \ldots, k$ robot i has to visit. For each $i \in \{1, 2, 3\}$, we initialize $n_{i,0} = 0$ and enforce for each $k \in \{1, \ldots, Z\}$:

$$(m_k = i \wedge n_{i,k} = n_{i,k-1} + 1) \vee (m_k \neq i \wedge n_{i,k} = n_{i,k-1}) \quad (12)$$

We keep the position variables $pos_{i,j}$ to store which zone is visited in step j of robot i, but their domain is slightly modified: knowing the number $n_{i,Z}$ of visits for each robot, the fictitious location $pos_{i,j} = -4$ is not needed anymore. Instead, we will simply disregard all $pos_{i,j}$ assigned for $j > n_{i,Z}$.

We also keep the variables $d_{i,j}$, but with a different meaning: $d_{i,j}$ stores the distance traveled by robot i from its $(j-1)$th position $pos_{i,j-1}$ to its jth position $pos_{i,j}$. We add the constraints (3) for defining the positions up to the start zone and the following constraints for $i \in \{1, 2, 3\}$ and $j \in \{1, \ldots, Z\}$:

$$\bigvee_{k=0}^{Z} \bigvee_{\substack{l=1 \\ l \neq k}}^{Z} \Big(pos_{i,j-1}{=}k \wedge pos_{i,j}{=}l \wedge d_{i,j} \geq D(k,l) \Big) \quad (13)$$

Note that instead of $d_{i,j} = D(k,l)$ we added inequalities because this improved the solving times, while minimization will anyways enforce equality (see also page 6).

A new variable d_i for $i \in \{1, 2, 3\}$ is used to store the total travel distance for each robot.

We enforce for $i \in \{1, 2, 3\}$:

$$(n_{i,Z} = 0 \wedge d_i \geq 0) \vee \bigvee_{k=1}^{Z} (n_{i,Z} = k \wedge d_i \geq \sum_{j=1}^{k} d_{i,l}) \quad (14)$$

which ensures that, if robot i has to visit k zones ($n_{i,Z} = k$) then its total travel distance d_i is (at least equal to) the sum of the distances traveled from $pos_{i,0}$ to $pos_{i,k}$. If robot i does not move at all (*i.e.*, $n_{i,Z} = 0$) then d_i will be (at least) zero.

In order to make sure that each robot visits all zones it has been assigned to (by means of variables m_k), we add the following constraint for each $i \in \{1, 2, 3\}$ and $k \in \{1, \ldots, Z\}$:

$$m_k = i \implies \bigvee_{j=1}^{Z} n_{i,Z} \geq j \wedge pos_{i,j} = k \quad (15)$$

which ensures that, if robot i is assigned to zone k then this zone will be visited at some step j (within the upper bound on the number of zones to be visited $n_{i,Z}$).

Furthermore, we introduce bounds on integer variables so that the solver can internally perform bit-blasting and represent them as bit-vectors. As a consequence, the following constraints were added for each $i \in \{1, 2, 3\}$ and $k, j \in \{1, \ldots, Z\}$:

$$1 \leq m_k \leq 3 \qquad 0 \leq n_{i,k} \leq Z \qquad 1 \leq pos_{i,j} \leq Z \quad (16)$$

Finally, we replace the nonlinear objective function specified in constraint (7) by a linear one: since all robots start from the start zone, we exploit symmetry and require an order on the total travel distances:

$$d_1 \geq d_2 \wedge d_2 \geq d_3 \quad (17)$$

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.

Z	F		F_1		F_2	
	Time	#solved	Time	#solved	Time	#solved
12	54.78	66/100	57.02	66/100	66.84	46/100

Table 2: Average solving time (sec) and #instances solved for encodings F, F_1 and F_2 on 100 benchmarks (TO: 2min).

	Opt1	Opt2	Opt3
Simple	107.10	157.85	166.57
Binary	117.40	104.69	95.55

Table 3: Optimization times (sec) for Z=12 zones using encoding C.

We can now minimize the total distance for the first robot:

$$\text{minimize } d_1 \tag{18}$$

Results. Table 1 shows a considerable improvement by encoding F over previous solutions for the selected benchmarks. In order to obtain statistically significant results, we also tested encoding F on 100 most recurring instances of the RCLL problem with 12 zones (see Table 2). Three optimizing solvers were used to carry out this series of experiments, namely `Z3`, `SMT-RAT` and `OptiMathSAT`. The latter specializes in optimization for real arithmetic while `SMT-RAT` is tuned for non-linear real arithmetic problems. However, our problem mostly involves optimization at the Boolean level and therefore the strengths of these two solvers could not be exploited to their fullest. For this reason, we report only results obtained using `Z3`.

Replacing a non-linear objective function with a linear one allowed us to reduce the complexity of the optimization problem at hand. Moreover, specifying bounds on integer variables in (16) and specifying lower bounds instead of equalities in (13) and (14) seemed to be also helpful. To analyze these potential sources of improvement, we made additional experiments with two variants of encoding F: in encoding F_1 we removed the bounds (16) for integer variables, and in encoding F_2 we replaced the inequalities in (13) and (14) by equalities (but the constraints (16) are kept in F_2). Table 2 shows the average results for the previously used 100 benchmarks. While working with unbounded integers does not seem to significantly affect the solving times (Fig. 4a), relaxing equalities had a stronger impact. Fig. 4b shows that the solving time for the encoding F_2 with equalities is almost always higher, and a fewer number of instances could be solved within the timeout (see Table 2).

5 Additional Notes

About the optimization algorithm. `Z3` adds the clauses shown in Eq. (2) to (i) exclude the current assignment S from the further search and (ii) improve the current upper-bound value μ [3]. However, as already noted in [17, 18], the inconsistency of the arithmetic constraint $f < \mu$ cannot be determined at the Boolean level. Therefore, inconsistencies on this term are not detected until the theory solver is invoked. Since the latter is more resource demanding and invoked less often, the performance of a solver based on linear-search optimization can be strongly affected when dealing with a complex and unstructured domain like the one at hand.

To examine these aspects in more detail, we implemented two basic optimization algorithms `Simple` and `Binary` with three different versions `Opt1`, `Opt2` and `Opt3` each, based on the satisfiability checking function of `Z3` (instead of its optimization function). For all versions of both algorithms, the input formula φ is constructed as in encoding C but without the optimization constraint (7).

For each found solution S with value ν for the objective function f, `Simple` adds a formula, similar to Equation (2) and described below, to reduce the further search to values from $[0, \nu)$ for f; note that for Equation (2) we would compute the optimal value μ under S and add $f < \mu$, however, for our problems ν will always be a local optimum as its value is defined by equations in encoding C. `Simple` terminates with the last found solution when the refined formula becomes unsatisfiable.

`Binary` determines first a solution with value ν for f and maintains in its further search lower and upper bounds l and u on the optimal value, which are initially 0 and ν, resp. In each further iteration, `Binary` first searches for a solution with an objective function value ν' in the lower half $[l, m)$ of the interval $[l, u)$. If a solution is found then u is set to ν'. Otherwise if there is a solution in the upper half $[m, u)$ with objective function value ν' then $l{:=}m$ and $u{:=}\nu'$ are set; if neither $[l, m)$ nor $[m, u)$ contain a solution then `Binary` terminates with the last solution.

For both algorithms, their three versions differ in the formula that is added for each found objective function value ν to restrict the further search. For `Simple`, version `Opt1` extends the formula φ by adding the constraint $f < \nu$. `Opt2` adds the formula $f < \nu \wedge \neg\varphi_{max}$, where φ_{max} encodes the sequence of visited zones for the robot with the longest travel distance. Finally, `Opt3` adds $f < \nu \wedge \neg\varphi_{max} \wedge \neg\varphi'_{max} \wedge \neg\varphi''_{max}$, where φ'_{max} and φ''_{max} are copies of φ_{max} with the index of the robot having the largest travel distance substituted by the indices of the other two robots. That means, `Opt3` adds further symmetry breaking constraints. For `Binary`, all three versions are analogous to what was previously described but additionally add the corresponding lower bounds on f.

Results. Table 3 shows results for the 12 zones bench-

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.

mark used for Table 1. The optimization algorithm in Z3 is (unsurprisingly) more efficient (81.64 sec, see encoding C for $Z = 12$ in Table 1) than our prototypical algorithms. Nevertheless, these algorithms allow us to compare different optimization alternatives. Especially, the results show that excluding the current solution on top of restricting the domain for the objective function value pays off only for the binary search Binary, but not for Simple.

Comparison to other planning methods. The results presented so far are encouraging as they show potential to increase scalability. However, the improved solving times are still high compared to other techniques. To illustrate this, we compare our results to traditional planning with PDDL, an action-centered language, inspired by STRIPS formulations of planning (see [8] for a complete description). PDDL planning, being heuristic-driven, is very fast but in general does not compute optimal solutions. For example, for the benchmark used in Table 1 for $Z = 12$ zones, the POPF planner [4] returns a schedule with objective function value 20.54 in 0.04 seconds, in contrast to the optimum 12.6 determined by OMT solving.

OMT-based methods are in general computationally more expensive than currently used planning techniques. Nevertheless, the optima found by OMT-based methods are far better than the solutions found by other planning techniques. The latter scale well in terms of efficiency, but give no guarantees on the quality of the computed solutions. We therefore investigated the effects of combining the two techniques. Given an instance of the RCLL, POPF is used first to compute an upper bound d_{max} on the maximum distance traveled by robots (as per Eq. (18)). Then, an SMT-LIB [2] encoding is created for the same instance and Z3 is called with the added constraint $d_1 \leq d_{max}$.

Results. Table 4 shows the effect of introducing the upper bound computed using planning. Results refer to encoding F without and with upper bound – the latter being called F_{UB}. At a first glance, it may look reasonable to conclude that having upper bounds on objective functions is beneficial: the solver could solve more instances and with an average time of 50.13 seconds, which is less than what was previously obtained. We therefore carried out Mann-Whitney U test on the 66 instances which were already solvable with encoding F to verify whether the above statement is statistically relevant. Mann-Whitney U test allows to test a null hypothesis that it is equally likely that a randomly selected value from one sample (solving times for F) will be less than or greater than a randomly selected value from a second sample (solving times for F_{UB}). Running this test, a p-value of 0.18 was obtained, which does not allow us to discard the null hypothesis within a reasonable significance level (*e.g.*, 5%).

Z	F		F_{UB}	
	Time	#solved	Time	#solved
12	54.78	66/100	50.13	70/100

Table 4: Average solving time (sec) and #instances solved for encodings F and F_{UB} (TO: 2min).

Further inspection of the results revealed that 27% of the instances which were already solvable with encoding F see an increase in solving time, with an average rising from 66.75 seconds to 78.87 seconds. We conjecture that reducing the search space over the reals does not necessarily lead to an improvement in solving time, as the number of feasible computation traces reduces therefore making the problem harder to solve. This point will be the subject of future investigations.

Model validation. In order to assess the feasibility and quality of the plans computed, we executed them in the RCLL simulation environment. We measured the time between start and end of the simulation (solving time excluded) for each plan; given that robots move at a speed between 0.1 and 0.5 (distance unit per time unit in simulation), a conservative estimate of the distance traveled can be obtained by multiplying the above mentioned time by, *e.g.*, 0.1. The ratio between this estimate and the solution computed by the solver resulted to be 1.42 ± 0.72. This difference is to be expected as planning assumptions are most often challenged during execution, due to unforeseen events and environment uncertainty – *e.g.*, when two robots get stuck or have to give way to one another. Such situations require the executive to adjust its current plan to cope with the unforeseen scenario, subsequently leading to (possibly) considerable variations. However, it should be noted that such variations occur irrespectively of the planning algorithm used.

6 Concluding Remarks

In this paper we have shown how SMT solving combined with optimization can be used to synthesize guaranteed-quality plans for robot fleets. Based on a RoboCup Logistics League case study, we highlighted weaknesses and strengths of our approach. The way how problems like the one presented are logically encoded has a strong influence on the solving time. In particular, we observed that improvements on optimization were gained by:

- applying symmetry breaking as much as possible;
- explicitly encoding transitive properties like for the monotonicity of sequences;

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.

- introducing bounds on integer-valued variables so that the solver can apply internal bit-blasting (however, adding bounds for real-valued variables does not always lead to improvements);
- avoiding non-linearities whenever possible;
- replacing equalities by inequalities if the objective function enforces the equality.

For our application the above suggestions led to improvements, and we expect similar effects for applications from other areas. Nevertheless, these observations cannot be generalized, because solver performances might be extremely sensitive to even minor changes in the encoding.

Concerning the solvers, while there exist efficient solvers for different types of optimization problems like combinatorial optimization or integer programming problems, there seem to be room for improvements for problems where the objective function is a real-arithmetic function but the search is over a finite set of objects, *i.e.*, where the problem seems to involve optimization in the arithmetic domain but at its core it is a purely combinatorial optimization problem.

Moreover, binary-search-based optimization algorithms seem to be very sensitive to the parameters used to drive the search. This point could be exploited, *e.g.*, to develop a portfolio approach, where different solvers, instantiated with different parameters, run in parallel.

Though our OMT-based planning module has been successfully integrated in the RCLL planning framework, several challenges are yet to be addressed in order to improve the performance when dealing with more complex tasks in the production phase. Future research will investigate ways to better exploit domain-specific knowledge, parallelisation strategies for optimization, and possible ways improve OMT by combining it with planning heuristics.

Acknowledgments

The authors would like to thank Igor Bongartz and Lukas Netz for their help with the implementation and validation of the results presented.

References

[1] E. Ábrahám and G. Kremer. Satisfiability checking: Theory and applications. In *Proc. of SEFM'16*, pages 9–23, 2016.

[2] C. Barrett, P. Fontaine, and C. Tinelli. The Satisfiability Modulo Theories Library (SMT-LIB). `www.SMT-LIB.org`, 2016.

[3] N. Bjørner, A. Phan, and L. Fleckenstein. νz - An optimizing SMT solver. In *Proc. of TACAS'15*, pages 194–199, 2015.

[4] A. J. Coles, A. Coles, M. Fox, and D. Long. Forward-chaining partial-order planning. In *Proc. of ICAPS'10*, pages 42–49, 2010.

[5] F. Corzilius, G. Kremer, S. Junges, S. Schupp, and E. Ábrahám. SMT-RAT: An open source C++ toolbox for strategic and parallel SMT solving. In *Proc. of SAT'15*, pages 360–368, 2015.

[6] N. T. Dantam, Z. K. Kingston, S. Chaudhuri, and L. E. Kavraki. Incremental task and motion planning: A constraint-based approach. In *Proc. of RSS'16*, 2016.

[7] L. M. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. of TACAS'08*, pages 337–340, 2008.

[8] M. Fox and D. Long. PDDL2.1: an extension to PDDL for expressing temporal planning domains. *J. Artif. Intell. Res. (JAIR)*, 20:61–124, 2003.

[9] T. Hofmann, T. Niemueller, J. Claßen, and G. Lakemeyer. Continual planning in Golog. In *Proc. of AAAI'16*, pages 3346–3353, 2016.

[10] S. Nedunuri, S. Prabhu, M. Moll, S. Chaudhuri, and L. E. Kavraki. SMT-based synthesis of integrated task and motion plans from plan outlines. In *Proc. of ICRA'14*, pages 655–662, 2014.

[11] T. Niemueller, E. Karpas, T. Vaquero, and E. Timmons. Planning competition for logistics robots in simulation. In *Proc. of PlanRob@ICAPS'16*, 2016.

[12] T. Niemueller, G. Lakemeyer, and A. Ferrein. Incremental task-level reasoning in a competitive factory automation scenario. In *Proc. of AAAI'13 Spring Symposium*, 2013.

[13] T. Niemueller, G. Lakemeyer, and A. Ferrein. The RoboCup Logistics League as a benchmark for planning in robotics. In *Proc. of PlanRob@ICAPS'15*, 2015.

[14] T. Niemueller, G. Lakemeyer, F. Leofante, and E. Ábrahám. Towards CLIPS-based task execution and monitoring with SMT-based decision optimization. In *Proc. of PlanRob@ICAPS'17*, to appear.

[15] R. Nieuwenhuis and A. Oliveras. On SAT modulo theories and optimization problems. In *Proc. of SAT'06*, pages 156–169, 2006.

[16] I. Saha, R. Ramaithitima, V. Kumar, G. J. Pappas, and S. A. Seshia. Automated composition of motion primitives for multi-robot systems from safe LTL specifications. In *Proc. of IROS'14*, pages 1525–1532, 2014.

[17] R. Sebastiani and S. Tomasi. Optimization in SMT with $\mathcal{LA}(\mathbb{Q})$ cost functions. In *Proc. of IJCAR'12*, pages 484–498, 2012.

[18] R. Sebastiani and S. Tomasi. Optimization modulo theories with linear rational costs. *ACM Trans. Comput. Log.*, 16(2):12:1–12:43, 2015.

[19] R. Sebastiani and P. Trentin. OptiMathSAT: A tool for optimization modulo theories. In *Proc. of CAV'15*, pages 447–454, 2015.

[20] R. Sebastiani and P. Trentin. Pushing the envelope of optimization modulo theories with linear-arithmetic cost functions. In *Proc. of TACAS'15*, pages 335–349, 2015.

[21] Y. Wang, N. T. Dantam, S. Chaudhuri, and L. E. Kavraki. Task and motion policy synthesis as liveness games. In *Proc. of ICAPS'16*, page 536, 2016.

Authorized licensed use limited to: Institute of Software. Downloaded on February 22,2025 at 14:24:50 UTC from IEEE Xplore. Restrictions apply.